

MAHARAJA INSTITUTE OF TECHNOLOGY MYSORE

An Autonomous Institute affiliated to VTU, Belagavi

Karnataka



Mini Project Report on

“STOCKWATCH: SMARTER INVENTORY AND SAFER AUDITS”

**In partial fulfillment of the requirements for the award of Degree of
MASTER OF COMPUTER APPLICATIONS**

Submitted By

NIRMITHA P – 4MH25MC063

SUCHITHRA R – 4MH25MC095

Under the Guidance of

Prof. Poornima H A

Assistant Professor

Department of MCA

MITM



DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

Maharaja Institute of Technology Mysore

**An Autonomous Institute affiliated to Visvesvaraya Technological University,
Belagavi**

Belawadi, Srirangapatna Tq, Mandya — 571477

2025-2026

MAHARAJA INSTITUTE OF TECHNOLOGY MYSORE

Department of Master of Computer Applications

An Autonomous Institute affiliated to VTU, Belagavi

Belawadi, Srirangapatna Tq, Mandya — 571477



Mini Project Certificate

This is to certify that **NIRMITHA P** bearing USN: **4MH25MC063** has successfully completed the Mini Project entitled **STOCKWATCH: SMARTER INVENTORY AND SAFER AUDITS** during the academic year 2025-26, in the II Semester of the Master of Computer Applications.

Signature of the Guide

Signature of the HOD

Signature of Internal

Signature of External

MAHARAJA INSTITUTE OF TECHNOLOGY MYSORE

Department of Master of Computer Applications

An Autonomous Institute affiliated to VTU, Belagavi

Belawadi, Srirangapatna Tq, Mandya — 571477



Mini Project Certificate

This is to certify that **SUCHITHRA R** bearing USN: **4MH25MC095** has successfully completed the Mini Project entitled **STOCKWATCH: SMARTER INVENTORY AND SAFER AUDITS** during the academic year 2025-26, in the II Semester of the Master of Computer Applications.

Signature of the Guide

Signature of the HOD

Signature of Internal

Signature of External

DECLARATION

I **NIRMITHA P** student of **2nd** semester MCA, Maharaja Institute of Technology Mysore, bearing USN: **4MH25MC063** hereby declare that the Mini Project entitled **STOCKWATCH: SMARTER INVENTORY AND SAFER AUDITS** has been carried out by me under the supervision of **Prof. POORNIMA H A, ASSISTANT PROFESSOR, DEPARTMENT OF MCA.**

I declare that this report has not been submitted to any other Organization/University for any award of degree or certificate.

Place: MIT Mysore

Date: 27-07-2026

NIRMITHA P

4MH25MC063

DECLARATION

I **SUCHITHRA R** student of 2nd semester MCA, Maharaja Institute of Technology Mysore, bearing USN: **4MH25MC095** hereby declare that the Mini Project entitled **STOCKWATCH: SMARTER INVENTORY AND SAFER AUDITS** has been carried out by me under the supervision of **Prof. POORNIMA H A, ASSISTANT PROFESSOR, DEPARTMENT OF MCA.**

I declare that this report has not been submitted to any other Organization/University for any award of degree or certificate.

Place: MIT Mysore

Date: 27-07-2026

SUCHITHRA R

4MH25MC095

ACKNOWLEDGEMENT

A mini project is an important part of academic learning, and its successful completion is possible with the guidance, encouragement, and support of many people. I take this opportunity to express my sincere gratitude to everyone who has helped and motivated us throughout the completion of this project.

I express my sincere gratitude to our respected Principal, **Dr. S. Murali**, for his constant support and encouragement.

I am also grateful to the Head of the Department, **Dr. Manjunath B**, for his invaluable guidance, encouragement, and suggestions throughout the course of this project.

Guidance and timely support play a crucial role in the successful completion of any project. I would like to extend my heartfelt thanks to **Prof. Poornima H. A**, my Project Guide, for her constant guidance, valuable suggestions, encouragement, and supervision throughout the development of this work.

I also express my sincere appreciation to the teaching and non-teaching staff of the Department of Master of Computer Applications for their cooperation, support, and assistance throughout this project.

My heartfelt thanks go to my parents for their unwavering support, encouragement, and motivation. Last but not least, I am thankful to my peers and friends for their continuous encouragement and support throughout this journey.

NIRMITHA P
4MH25MC063

ACKNOWLEDGEMENT

A mini project is an important part of academic learning, and its successful completion is possible with the guidance, encouragement, and support of many people. I take this opportunity to express my sincere gratitude to everyone who has helped and motivated us throughout the completion of this project.

I express my sincere gratitude to our respected Principal, **Dr. S. Murali**, for his constant support and encouragement.

I am also grateful to the Head of the Department, **Dr. Manjunath B**, for his invaluable guidance, encouragement, and suggestions throughout the course of this project.

Guidance and timely support play a crucial role in the successful completion of any project. I would like to extend my heartfelt thanks to **Prof. Poornima H. A**, my Project Guide, for her constant guidance, valuable suggestions, encouragement, and supervision throughout the development of this work.

I also express my sincere appreciation to the teaching and non-teaching staff of the Department of Master of Computer Applications for their cooperation, support, and assistance throughout this project.

My heartfelt thanks go to my parents for their unwavering support, encouragement, and motivation. Last but not least, I am thankful to my peers and friends for their continuous encouragement and support throughout this journey.

SUCHITHRA R
4MH25MC095

ABSTRACT

Inventory management is an important part of organizations such as colleges, warehouses, and companies. It helps in tracking inventory items and maintaining appropriate stock levels. Many organizations still rely on manual methods such as registers or spreadsheets, which can lead to errors, data loss, and inefficient inventory tracking. To overcome these challenges, an automated inventory monitoring system is required.

The project, "**StockWatch: Smarter Inventory and Safer Audits**," is a web-based Inventory Monitoring System developed using Python, Flask, REST APIs, and MySQL. The frontend is developed using HTML, CSS, and JavaScript. The main objective of the system is to provide accurate inventory tracking, improve transparency, and simplify inventory management processes.

The system enables users to manage inventory items, update stock quantities, and monitor stock movements in real time. Every inventory transaction is recorded through audit logs, ensuring complete traceability of inventory activities. The system supports role-based access control for administrators and department users, providing secure and controlled access to inventory data. Approval mechanisms are also included for selected operations to improve accountability and operational control.

StockWatch incorporates automatic low-stock monitoring and alert generation to help users prevent inventory shortages. The system also includes QR code-based stock identification and updates, enabling faster inventory operations while reducing manual data entry errors. To further enhance monitoring and security, the project integrates a machine learning-based anomaly detection module using the Isolation Forest algorithm from Scikit-learn. This module analyzes inventory transaction patterns, identifies unusual stock activities, and generates alerts for administrative review.

Communication between the frontend and backend is carried out through REST APIs, providing a structured, scalable, and maintainable architecture. Overall, StockWatch improves inventory management by reducing manual effort, increasing data accuracy, enhancing security, and providing better control over inventory operations. The system offers a reliable solution for organizations seeking efficient inventory monitoring, audit tracking, and intelligent anomaly detection.

TABLE OF CONTENTS

Chapter No.	Content	Page No
1	Introduction	1-4
2	Literature Survey	5-8
3	Methodology	9-16
4	System Design	17-18
5	System Architecture	19-20
6	Detailed Design	21-26
7	Implementation	27-29
8	System Testing	30-32
9	Snapshots	33-37
10	Conclusion	38
11	Future Enhancement	39
12	References	40

CHAPTER 1

INTRODUCTION

Inventory management plays a very important role in every organization, industry, institution, and business. It helps organizations maintain proper records of materials, equipment, and other assets. An effective inventory management system ensures that the right quantity of items is available at the right time while reducing unnecessary stock and operational costs. However, many organizations still depend on manual methods or basic software tools for inventory tracking, which can lead to errors, delays, and difficulties in monitoring stock movements.

With the growth of digital technologies, organizations are moving towards automated inventory management solutions. Modern inventory systems provide better control over stock levels, improve data accuracy, and support faster decision-making. Studies show that web-based inventory systems improve inventory control, while QR code-based warehouse management systems enable faster and more accurate inventory operations by reducing manual data entry [1][2].

In many organizations, inventory activities involve multiple departments and users. Every stock addition, removal, transfer, or update must be recorded properly to maintain accountability. Without proper monitoring, organizations may face problems such as missing inventory, unauthorized stock movements, inaccurate records, and difficulties during audits. Audit logs help maintain records of system activities, while anomaly detection techniques can identify unusual inventory transactions and improve system security [3][4].

Real-time inventory visibility has also become important in modern inventory management. It enables organizations to make informed decisions, avoid stock shortages, and improve operational efficiency through continuous monitoring [5]. Despite these advancements, many existing inventory systems focus only on basic inventory functions and do not integrate features such as audit logging, QR code support, and anomaly detection into a single platform.

To address these challenges, the proposed project, “**StockWatch: Smarter Inventory and Safer Audits,**” is developed as a web-based Inventory Management System. The system manages inventory records, tracks stock movements, maintains detailed audit logs, generates alerts, supports QR code-based stock updates, and uses anomaly detection techniques to

identify unusual inventory activities. By combining these features into a single platform, StockWatch aims to improve inventory accuracy, transparency, accountability, security, and overall operational efficiency.

1.1 OBJECTIVE

The main objective of the proposed project, “**StockWatch: Smarter Inventory and Safer Audits,**” is to develop a web-based Inventory Management System that helps organizations manage and monitor their inventory efficiently. The system aims to maintain accurate inventory records, track stock movements, and provide better visibility of inventory activities. It is designed to reduce manual errors and improve the overall efficiency of inventory management processes.

Another objective of the project is to improve transparency and accountability by maintaining detailed audit logs of all inventory transactions and user activities. The system also supports QR code-based stock updates for faster and more accurate inventory operations. In addition, it generates alerts for unusual inventory activities and uses anomaly detection techniques to identify suspicious stock movements. By combining these features into a single platform, the project aims to improve inventory control, simplify auditing, and support better decision-making within an organization.

1.2 SCOPE OF WORK

The scope of the proposed project, “**StockWatch: Smarter Inventory and Safer Audits,**” is to provide a centralized platform for managing inventory operations within an organization. The system maintains accurate inventory records, tracks stock movements, and supports department-wise inventory management. It allows users to add, update, and monitor inventory items while maintaining complete transaction records.

The system supports QR code-based inventory operations to reduce manual data entry and improve accuracy. Stock activities such as addition, removal, and transfer are recorded for future reference. Detailed audit logs help administrators track user actions and inventory changes, making the auditing process easier and more reliable.

The system also uses anomaly detection to identify unusual inventory activities and generates alerts for abnormal movements and low stock levels. Approval workflows ensure that important transactions are properly reviewed. The project is developed using **Flask, MySQL, HTML, CSS, and JavaScript**. Future improvements may include mobile support, IoT-based monitoring, predictive stock analysis, and advanced reports.

1.3 MOTIVATION

Inventory management is an important activity in every organization, but many organizations still depend on manual processes or basic systems to manage inventory records. These methods can lead to data-entry errors, inaccurate stock records, delays in updating information, and difficulties in tracking inventory movements. Such problems can affect operational efficiency and make inventory management more difficult [1].

Another motivation for this project is the need for better transparency and accountability in inventory operations. Without proper tracking, it becomes difficult to identify unauthorized stock movements or maintain accurate transaction records. Maintaining audit logs and monitoring inventory activities can help organizations control their resources and simplify the auditing process [3].

Technologies such as QR codes and anomaly detection provide opportunities to improve inventory management. QR codes can reduce manual work and improve inventory accuracy [2], while anomaly detection can identify unusual activities using transaction and log data [4]. Therefore, StockWatch combines inventory management, stock tracking, audit logging, and anomaly detection in a single platform to improve efficiency, transparency, and accountability.

1.4 PROBLEM DEFINITION

Organizations often face difficulties in managing inventory because of the increasing number of items, transactions, and users involved in daily operations. Maintaining accurate records of stock additions, removals, and transfers becomes difficult when inventory information is stored in different records or updated manually. This can result in stock mismatches, missing details, duplicate entries, and limited visibility of inventory activities [1].

Another major problem is the lack of proper tracking and monitoring of inventory transactions. Organizations may find it difficult to identify when a stock movement occurred, who performed it, and whether it was authorized. This creates problems during inventory verification and auditing. Unusual activities such as excessive stock withdrawals, unexpected stock reductions, or irregular transaction patterns may also remain unnoticed without continuous monitoring [3][4].

Therefore, there is a need for a centralized system that can manage inventory records, track stock movements, maintain transaction history, monitor user activities, and identify unusual inventory behaviour. Such a system can improve inventory control, simplify auditing, and ensure better use of inventory resources.

CHAPTER 2

LITERATURE SURVEY

Inventory management is an important part of every organization because it helps in maintaining stock records and controlling inventory efficiently. Many researchers have developed different inventory management systems to reduce manual work, improve stock tracking, and increase accuracy. Technologies such as web applications, QR codes, and anomaly detection have made inventory management faster and more reliable. This chapter discusses the existing inventory management systems, their features, and their limitations. It also explains the research gap and how the proposed StockWatch: Smarter Inventory and Safer Audits provides a better solution by combining inventory monitoring, audit logging, QR code support, and anomaly detection in a single system.

2.1 LITERATURE REVIEW

[1] “Web-Based Systems for Inventory Control in Organizations: A Systematic Review”

Authors: Gunther Misahuaman, Dr. Alfredo Daza and Emily Zavaleta.

This paper reviews web-based inventory systems developed between 2016 and 2021. It explains that web-based systems improve inventory tracking, accuracy, reporting and efficiency. However, it does not discuss audit logs and anomaly detection.

[2] “QR Based Web Application for Warehouse Management System”

Authors: Michael Siek and Jefferson Richard Djaya.

This paper presents a warehouse management system that uses QR codes for receiving, issuing and tracking goods. It provides faster data entry, real-time stock updates and fewer manual errors. StockWatch uses similar QR-based tracking along with audit logs and anomaly detection.

[3] “eAudit: A Fast, Scalable and Deployable Audit Data Collection System”

Authors: R. Sekar, Hanke Kimm and Rohit Aich.

This paper presents eAudit, a system for collecting audit logs efficiently. It reduces performance issues, storage requirements and the risk of losing or modifying log data. The study shows the importance of secure and reliable audit records.

[4] “A Data-Driven Framework for Anomaly Detection in Industrial Systems Using Log Data”

Authors: Merle Hewing, Yuanchen Zhao, Manuel Vossel, Paul Nieschler and Tobias Kleinert.

This paper presents a framework that uses system logs to detect unusual behaviour and failures. Log data is more easily available and less expensive than sensor data. The framework improves system monitoring and helps users understand the causes of problems.

[5] “AI-Driven Framework for Real-Time Financial Anomaly Detection and Transparent Audit Trail Generation in ERP Systems”

Author: Kulasekhara Reddy Kotte.

This paper proposes an AI-based framework for real-time anomaly detection and secure audit trail generation. It studies past transactions to identify unusual activities and maintains protected audit records. The study shows that AI monitoring can improve security and accountability.

[6] “Enhancing Warehouse Inventory Management Through IoT Tools for Monitoring Stock Items”

Authors: Sunaryo Winardi, Arifin, Ng Poi Wong, Apriyanto Halim and Sunario Megawan.

This paper presents an IoT-based system for monitoring warehouse stock in real time. It uses sensors, cloud services and wireless communication to improve stock visibility and decision-making. However, IoT systems may involve higher costs and integration difficulties.

[7] “Log-Based Anomaly Detection: Relation Between the Logs”

Authors: Sneha Sunil, Arvind Suresh and Hemamalini V.

This paper uses deep learning to identify relationships between system log events. It helps detect abnormal behaviour without requiring time-consuming manual log analysis. The method improves system monitoring, security and reliability.

[8] “Log-Based Anomaly Detection of Enterprise Software: An Empirical Study”

Authors: Nadun Wijesinghe and Hadi Hemmati.

This paper compares DeepLog, LogAnomaly, LogBERT and LSTM anomaly detection models. The results show that model performance depends on the structure and type of log data. The study also recommends time-based data splitting for more realistic testing.

[9] “Optimizing Web-Based Inventory Management System Using QR Code Technology”

Authors: Tedla Balaji, Padidi Ganesh, Vasanthula Hari, Prudhvi Arupya and Shaik Lathifunnisa.

This paper presents a web-based inventory system in which each item has a unique QR code. QR codes support quick access to item details, automatic updates and real-time tracking. The system improves efficiency and reduces errors compared to manual methods.

[10] “Enhancing Inventory Management with Progressive Web Applications (PWAs): A Scalable Solution for Small and Large Enterprises”

Author: Abhi Desai.

This paper proposes a PWA-based inventory system with QR scanning, user management and offline access. PWAs provide cross-platform support and lower maintenance costs. Its ideas about QR codes and web access are relevant to StockWatch.

2.2 RESEARCH GAP

Research in inventory management and anomaly detection has increased with the growth of digital systems. Existing studies focus on inventory tracking, QR codes, audit logs and anomaly detection. However, most studies examine these features separately and do not provide a complete Inventory Management System. Based on the literature review, the following research gaps were identified:

- Lack of anomaly detection models designed specifically for inventory transactions.
- Weak integration between audit logs and machine-learning-based anomaly detection.
- Lack of lightweight, real-time inventory systems for small and medium organizations.
- Limited explanation of why a transaction is identified as unusual.

- Poor handling of human errors in inventory transaction records.
- Lack of systems combining QR codes, audit logs and anomaly detection.
- Lack of department-wise anomaly detection in inventory systems.
- Limited research using small and medium-sized inventory datasets.
- Lack of automatic alerts connected to anomaly detection.
- Lack of complete inventory lifecycle coverage, including item entry, stock movement, auditing, anomaly detection and reporting.
- Limited focus on secure and protected audit records in inventory systems.
- Limited real-world testing and deployment of academic inventory monitoring systems.

CHAPTER 3

METHODOLOGY

The proposed system, **StockWatch: Smarter Inventory and Safer Audits**, is a web-based Inventory Management System. It is developed using Python, Flask, MySQL, HTML, CSS, and JavaScript. The system provides a centralized platform to manage inventory records, stock movements, purchase orders, requests, alerts, audit logs, and unusual inventory activities.

StockWatch follows a modular design and contains four user modules: Admin, Inventory Manager, Department Head, and Auditor. Each user has a separate dashboard and can access only the functions allowed for their role.

1.Admin Module

The Admin has full control over the system. The Admin can register users, update user details, and block or unblock user accounts. The Admin can also view audit logs and run the machine-learning anomaly scan to identify unusual inventory activities.

2.Inventory Manager Module

The Inventory Manager handles the main inventory operations. The manager can add new items, update item details, search for items, and deactivate old items. Stock movements such as IN, OUT, TRANSFER, and ADJUSTMENT can also be recorded.

The Inventory Manager can create purchase orders, approve pending orders, cancel orders, and record the receipt of items from suppliers. When an order is received, the system updates the stock quantity and records an IN movement.

3.Department Head Module

The Department Head can view available inventory items and their current quantities. They can raise stock requests by entering the required item, quantity, and purpose. They can also view the status of their requests to check whether they are pending, approved, or rejected.

4.Auditor Module

The Auditor has read-only access to the system. The Auditor can view inventory items, categories, suppliers, stock movements, alerts, and audit logs. They can also run the anomaly detection scan. The Auditor cannot add, update, or delete records, which helps maintain a proper division of responsibilities.

5. Stock Movement and Audit Process

Whenever an IN, OUT, TRANSFER, or ADJUSTMENT operation is performed, the inventory quantity is updated and the transaction is recorded in the stock movement module. The record contains details such as the item, movement type, previous quantity, new quantity, user, reason, and date.

Important user actions are also recorded in the audit log. Each log contains details such as the user, module, action, affected record, IP address, and date. This creates a clear history of system activities and makes auditing easier.

6. Low-Stock and Anomaly Detection

The system continuously checks inventory quantities. When the quantity of an item reaches or falls below its predefined level, a low-stock alert is generated. This helps the Inventory Manager take action before the item becomes unavailable.

The anomaly detection module uses the Isolation Forest algorithm to analyse stock movement data. It identifies unusual activities such as sudden stock increases, unexpected reductions, or abnormal transaction quantities. When an anomaly is found, the system generates an alert for further investigation.

7. QR Code Support

The system generates a unique QR code for each inventory item. The QR code helps users identify an item and access its information quickly. This reduces manual searching and supports faster inventory operations.

8. Backend Integration and Data Handling

All modules communicate with a centralized MySQL database through Flask API endpoints. Relational tables connect users, departments, suppliers, categories, inventory items, purchase orders, invoices, stock movements, approvals, alerts, and audit logs. This structure supports accurate storage, easy retrieval, and complete tracking of inventory information.

9.Server Environment

The Flask backend runs locally at <http://127.0.0.1:5000> and processes API requests, database operations, and the machine-learning algorithm. MySQL is managed through phpMyAdmin. The system can be accessed through a web browser on an authorized computer.

10.Frontend and Security

The user interface is developed using HTML, CSS, and JavaScript. It provides separate pages and options based on the logged-in user's role. The system uses JSON Web Tokens to manage login sessions and control access.

Passwords are stored in hashed form. Input validation is applied to forms to prevent incomplete or invalid data, negative quantities, SQL injection, and Cross-Site Scripting attacks.

11.Scalability and Performance

The modular design allows new features to be added without making major changes to the existing system. Future features may include email notifications, barcode scanning, mobile support, predictive stock analysis, and automated supplier orders. The lightweight design allows the application to work on desktops, laptops, and tablets.

The overall workflow of StockWatch is:

User Login → Role-Based Dashboard → Inventory Operation → Stock Movement Recording → Quantity Update → Audit Log Generation → Low-Stock Monitoring → Anomaly Detection → Alert Generation → Dashboard and Reports

This methodology provides accurate inventory tracking, controlled access, better monitoring, improved security, and easier auditing.

3.1 DATA FLOW DIAGRAMS

ZERO LEVEL DATA FLOW DIGRAM

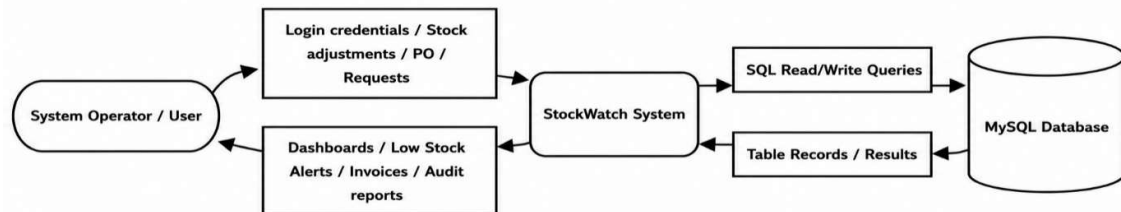


Fig 3.1.1: Zero level DFD: Provides a high-level overview of the StockWatch system and its interaction with external users. It shows the overall flow of information between users and the system.

FIRST LEVEL DATA FLOW DIAGRAM

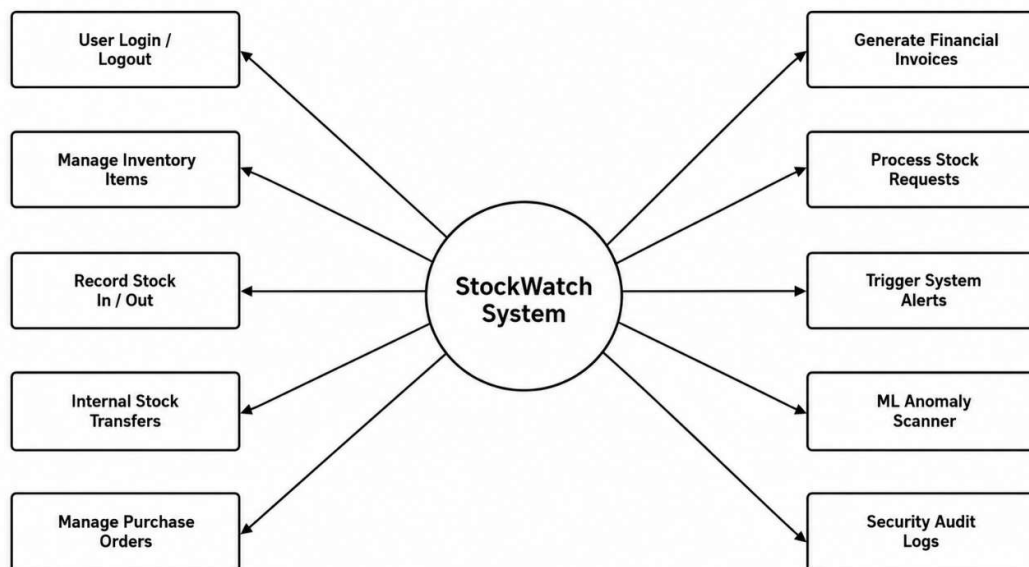


Fig 3.1.2: First level DFD: Illustrates the major functional processes of the StockWatch system. It explains how inventory data is processed and stored within different modules.

SECOND LEVEL DATA FLOW DIAGRAM

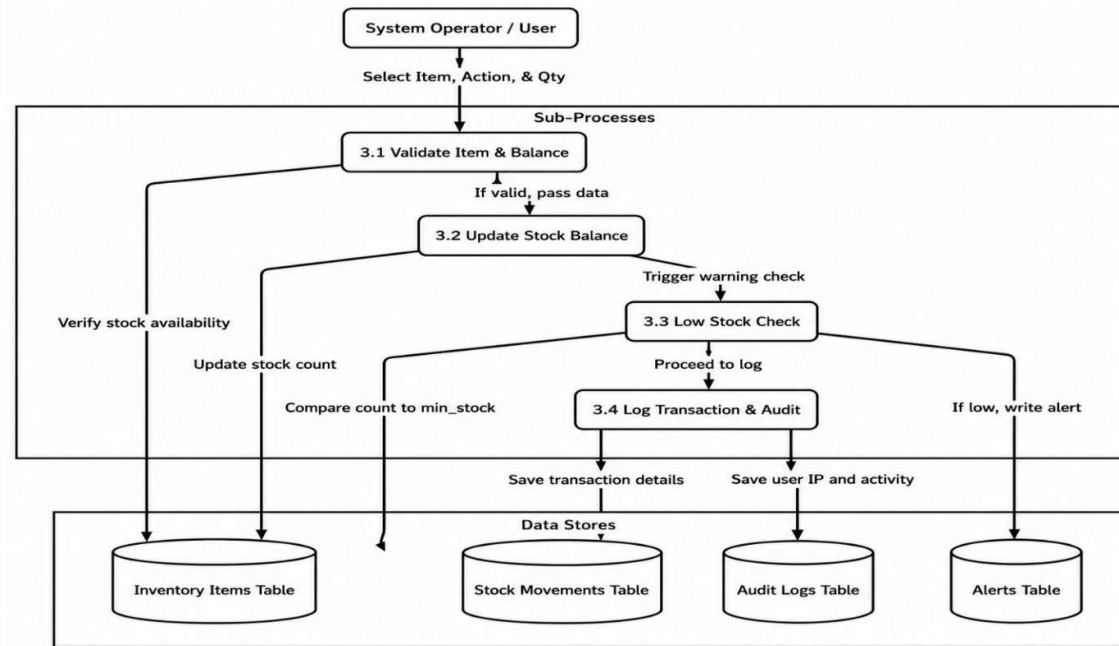


Fig 3.1.3: Stock Movements Details: Represents the workflow for stock addition, removal, transfer, and adjustment operations. It shows how inventory quantities are updated after each transaction.

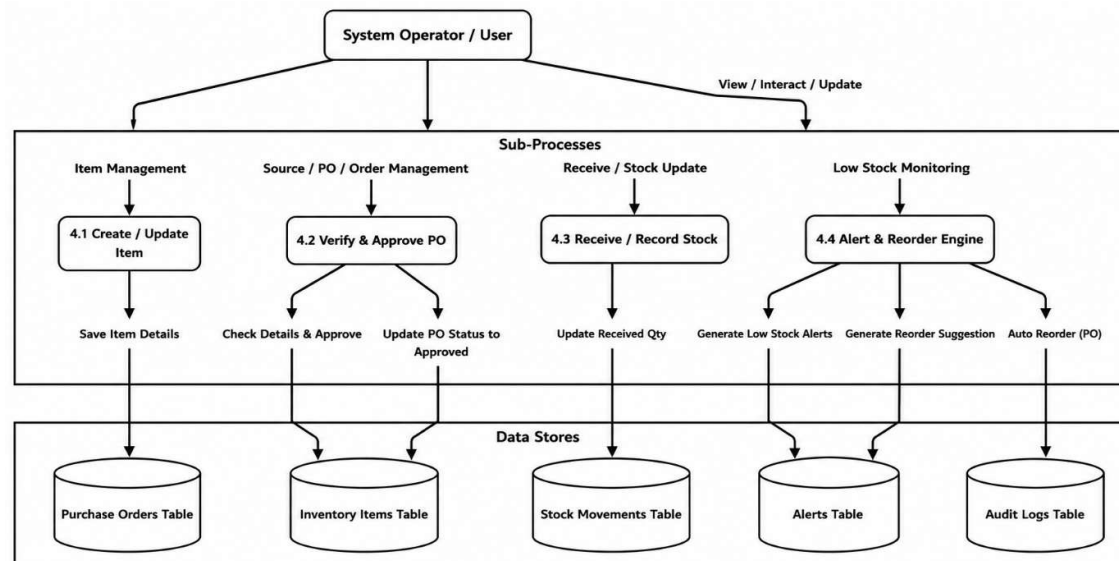


Fig 3.1.4: Purchase orders details: Shows the complete purchase order process from creation to approval and stock receipt. It also illustrates how purchase records are maintained in the system.

3.2 TOOLS AND TECHNOLOGY

Tool Name	Description
Python	Python is used to develop the main application logic of StockWatch. It processes inventory operations, stock movements, purchase orders, approvals, alerts, audit logs, and anomaly detection.
Flask	Flask is a lightweight Python web framework used to create the StockWatch application and its API routes. It receives requests from the user interface, processes them, communicates with the database, and returns the required response.
MySQL and phpMyAdmin	MySQL is used to store users, departments, suppliers, categories, inventory items, purchase orders, stock movements, approvals, alerts, invoices, and audit logs. phpMyAdmin provides a web interface for creating tables, running SQL queries, and managing database records.
HTML5, CSS3 and JavaScript	HTML5 is used to create the structure of forms, tables, dashboards, and other pages. CSS3 is used to design the layouts, colours, buttons, cards, and responsive screens. JavaScript handles form validation, API communication, page navigation, QR code scanning, and displaying data received from the Flask server.
Scikit-learn	Scikit-learn is a Python machine-learning library used to implement the Isolation Forest algorithm. The algorithm analyses stock movement data and identifies unusual transactions, such as unexpected stock increases or reductions.
html5-qrcode	html5-qrcode is a JavaScript library used to access the device camera and scan QR codes through the web browser. It allows users to identify inventory items and retrieve their information quickly, reducing manual searching and data entry.

3.3 SYSTEM REQUIREMENTS SPECIFICATION

HARDWARE REQUIREMENTS

Component	Minimum Requirement	Recommended Requirement
Processor	Intel / AMD Dual Core 2.0 GHz	Intel i3 or higher
RAM	4 GB	8 GB or higher
Hard Disk	10 GB free space	20 GB free space
Monitor	15" Color Monitor	LED Monitor
Keyboard & Mouse	Standard Keyboard and Mouse	Standard USB devices
Camera	USB Webcam (for QR scanning)	Integrated HD Webcam
Network Connection	Localhost or LAN	Localhost with internet (for chart CDN links)

SOFTWARE REQUIREMENTS

Software Component	Details / Purpose
Operating System	Windows 10/11 or Linux (Ubuntu/CentOS)
Web Server	Python built-in server (Flask) and Apache (included in XAMPP)
Languages	Python 3.x, HTML5, CSS3, and JavaScript (ES6)
Database	MySQL Server (managed through phpMyAdmin/XAMPP)
Development Environment	XAMPP for the database and Python virtual environment (venv) for libraries

STOCKWATCH: SMARTER INVENTORY AND SAFER AUDITS

Software Component	Details / Purpose
Web Browsers	Google Chrome, Microsoft Edge, or Mozilla Firefox
Editor / IDE	Visual Studio Code or PyCharm

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM PERSPECTIVE

The StockWatch system is designed as a modular, web-based program that connects a clean frontend user interface with a secure backend database. Built on a client-server architecture, the system allows users to open and run the application in a standard web browser without needing to download or install any software on their individual computers. All heavy tasks, database changes, and security checks are handled on the server, which keeps the system fast and easy to maintain.

From a system perspective, StockWatch links four key stakeholders: the Admin, the Inventory Manager, the Department Head, and the Auditor. These users log in and access their respective dashboards to manage items, record stock movements, request items, and view logs. Because the app has clear boundaries, each role is kept inside its own module and cannot access screens or run operations they are not allowed to do.

The database layer acts as the backbone of the entire system. It holds all the persistent data, such as items lists, category links, suppliers, purchase orders, invoices, and logs. By keeping all this data in one central MySQL database, we prevent data duplication and make sure that any stock changes are immediately updated across all modules. If an item is added or received in the warehouse, the new stock balance is instantly visible on both the manager's and the auditor's screens.

By using this design structure, StockWatch separates the user screens from the database rules. This modular setup makes the app highly scalable, meaning we can add new features—like automated email alerts, barcode scanning, or physical scanner hardware integrations—in future updates without having to rewrite the main database structure.

4.2 CONTEXT DIAGRAM OF THE SYSTEM

The Context Diagram represents the entire StockWatch application as a single central process. It shows the boundaries of the system by illustrating how data moves back and forth between

the app, the external users (Admin, Manager, Head, Auditor), and the MySQL database. It acts as a bird's-eye view of all inputs and outputs without showing the detailed inner workings of the code.

In the diagram, StockWatch is placed in the center. The users interact with the app by sending inputs such as login details, item registrations, stock adjustments (IN/OUT), purchase orders, and issue requests. The system processes these inputs and sends back outputs like dashboard charts, low stock alerts, printable bills, and activity reports. The database is shown as a separate external box that constantly reads and writes records based on the user actions.

By looking at this context diagram, stakeholders and developers can quickly understand the scope of the project. It clearly shows who can talk to the system, what data they need to provide, and what information they will receive in return. This diagram makes it easy to visualize the complete flow of data.

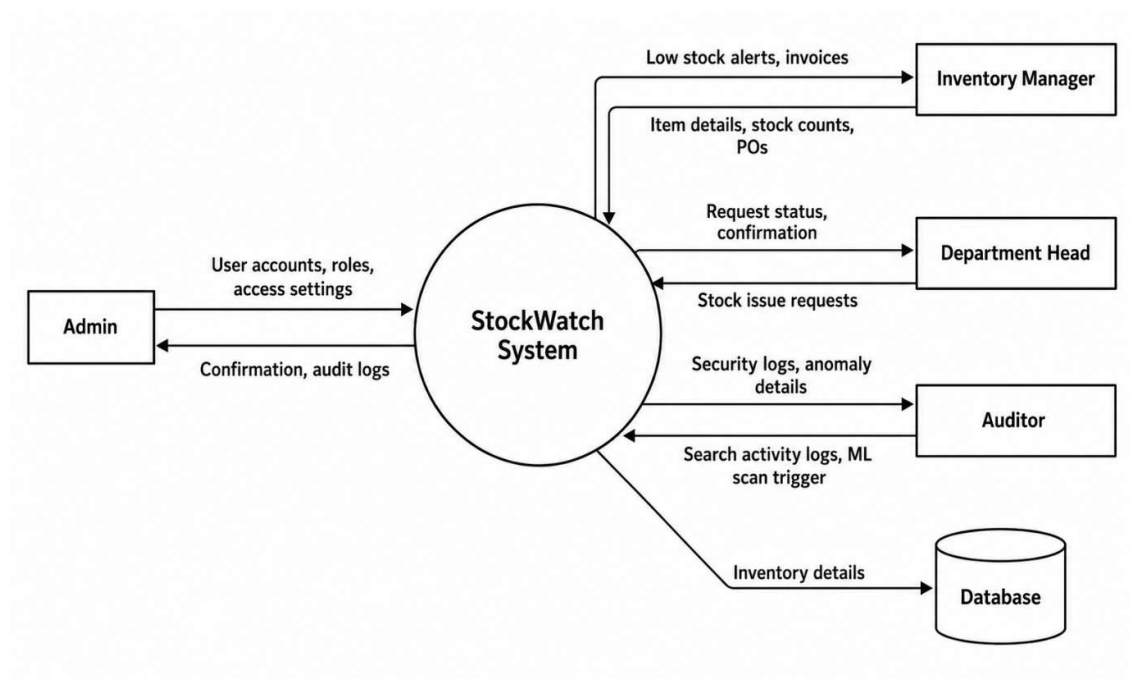


Fig 4.2: Context diagram of the system: Presents the overall interaction between users, the StockWatch system, and the database. It helps in understanding the system boundaries and external data flow.

CHAPTER 5

SYSTEM ARCHITECTURE

The StockWatch system follows a standard three-tier web-based architecture. This architecture consists of the **presentation layer (front end)**, the **application layer (business logic)**, and the **data layer (database)**. Splitting the system into these three layers ensures secure operations, easy updates, and a clear separation of tasks.

At the **presentation layer**, the system uses HTML, CSS, and JavaScript to provide dynamic, user-friendly screens for the Admin, Inventory Manager, Department Head, and Auditor. Each user logs into their specific dashboard where they can run authorized tasks. For example, managers can manage items and view purchase orders, while department heads can submit issue requests. All these screens run directly inside a standard web browser, meaning users do not need to install any special software.

The **application layer** contains the business rules and core logic written in Python using the Flask framework. This layer processes all incoming requests from the frontend, checks user login tokens (JWT) to enforce role security, and performs calculations. For example, when a department head requests stock, the application layer checks the database to see if enough items are available and approves or rejects the request based on the count. It also contains the Isolation Forest algorithm used to scan transactions for outliers.

The **data layer** uses MySQL to manage all persistent information. This includes user credentials, item quantities, reorder parameters, stock movement histories, approvals, invoices, and audit logs. The database is designed with foreign keys to ensure data consistency and prevent errors. It allows quick data retrieval when a user searches for items or runs reports.

This three-tier setup provides several benefits. It makes the system easy to maintain because updates in one layer (for example, redesigning the login form on the frontend) do not affect the backend code or database tables. It also increases security by blocking the frontend from directly talking to the database. Additionally, it makes the app highly scalable, allowing new features like automated supplier emails or barcode scanning to be added in future versions with minimal disruption.

The system architecture can be summarized as follows:

- **Users** (Admin, Manager, Head, Auditor) interact via web browsers.
- **Front-End** (HTML/CSS/JS) displays dashboards and forms.
- **Business Logic** (Python Flask scripts) validates inputs and executes operations.
- **Database Layer** (MySQL) stores and retrieves data.

By adopting this architecture, the StockWatch system ensures a robust and secure foundation for reliable inventory tracking.

CHAPTER 6

DETAILED DESIGN

The detailed design phase of the StockWatch system refines the high-level architecture into specific models and diagrams. This stage explains the internal structure of the database, how users interact with the app, and the system functionalities. Two core tools used in this phase are the **Entity-Relationship (ER) Diagram** and the **Use Case Diagrams**, which together show the system's database design and functional behavior.

6.1 ENTITY RELATIONSHIP (ER) DIAGRAM

Entity-Relationship Diagram for StockWatch models the logical structure of the database and shows the relationships among its main entities. The center of the system is the **inventory_items** table, which stores details of every cataloged item, including the item code, item name, current quantity, minimum stock level, reorder alert level, reorder quantity, unit of measurement, unit price, and unique QR code string. This table is linked to the **users** table, which stores the operator accounts responsible for stock entries. It also connects to the **departments** table to assign items to specific areas, the **categories** table to organize stock types, and the **suppliers** table to manage vendor contact information.

The **stock_movements** table records every inventory entry (IN), usage (OUT), transfer, or manual adjustment, linking back to the items and the performing operator user. The **purchase_orders** table stores vendor purchase requests, linked to **purchase_order_items** for line-by-line items, and automatically creates a billing statement in the **invoices** table when received. The **approvals** table tracks departmental issue requests raised by users, and the **audit_logs** table records all database changes with user IDs and client IP addresses for tracking. This relational structure ensures data consistency, integrity, and quick retrieval across all modules.

The database schema connects the entities as follows:

- **Users** belong to a **Department**.
- **Inventory Items** belong to a **Category** and have a **Supplier**.
- **Stock Movements** record changes to **Inventory Items** and are performed by a **User**.

- **Purchase Orders** are sent to a **Supplier** and contain multiple **Purchase Order Items**.
- **Invoices** are created from a **Purchase Order** and contain **Invoice Items**.
- **Approvals** track requests for **Inventory Items** raised by a **User**.
- **Audit Logs** record actions performed on any table by a **User**.

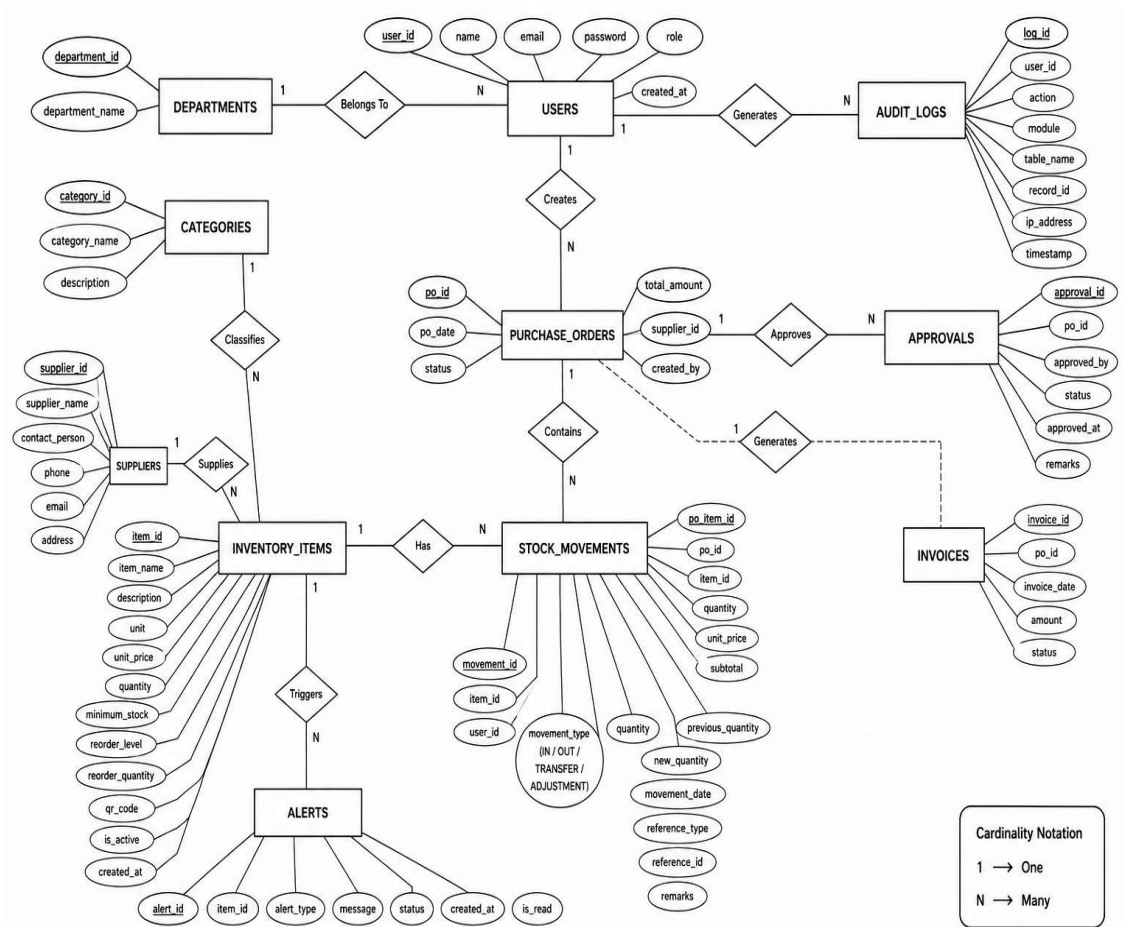


Fig 6.1: Entity-Relationship (ER) Diagram: Shows the relationships among the database entities used in the StockWatch system. It provides a clear view of how inventory data is organized and connected.

6.2 USE CASE DIAGRAMS

The Use Case Diagram shows the interactions between the system's key actors (Admin, Inventory Manager, Department Head, Auditor) and the StockWatch functions. It provides a visual map of who has permissions to run each action in the application.

- **Admin Actor:** Manages accounts, blocks/unblocks operators, and views audit logs.
- **Inventory Manager Actor:** Controls items, records stock logs, creates and receives purchase orders.
- **Department Head Actor:** Logs in and raises stock issue requests.
- **Auditor Actor:** Performs read-only security reviews, views alerts, and triggers anomaly scans.

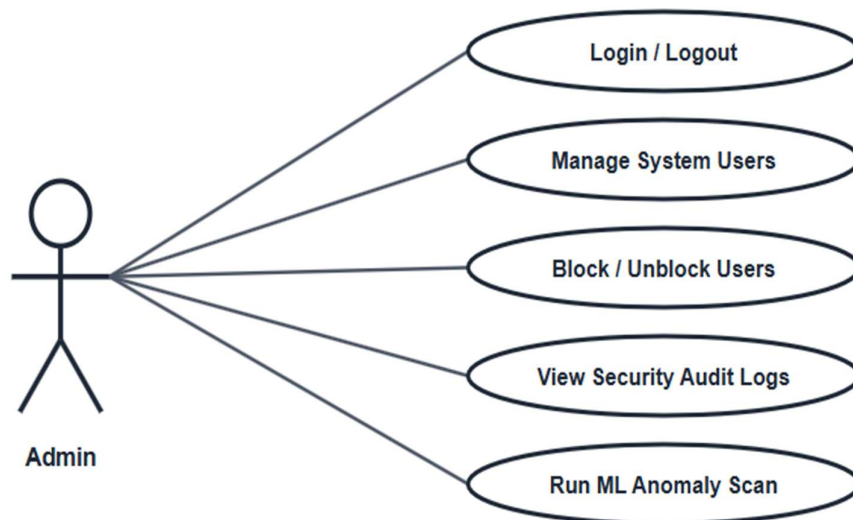


Fig 6.2.1: Admin Module Use Case: Illustrates the operations performed by the Admin, including user management and system monitoring. It defines the responsibilities assigned to the administrator.

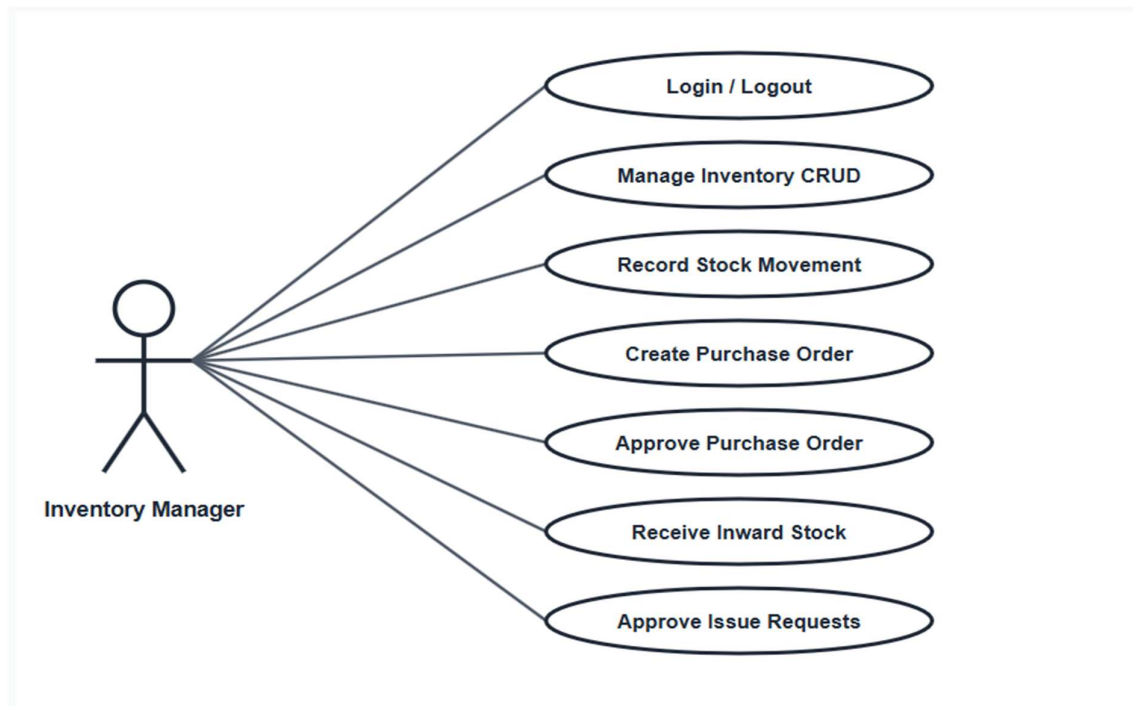


Fig 6.2.2: Inventory Manager Use Case: Shows the activities carried out by the Inventory Manager, such as managing inventory, stock movements, and purchase orders. It defines the permissions available for inventory operations.

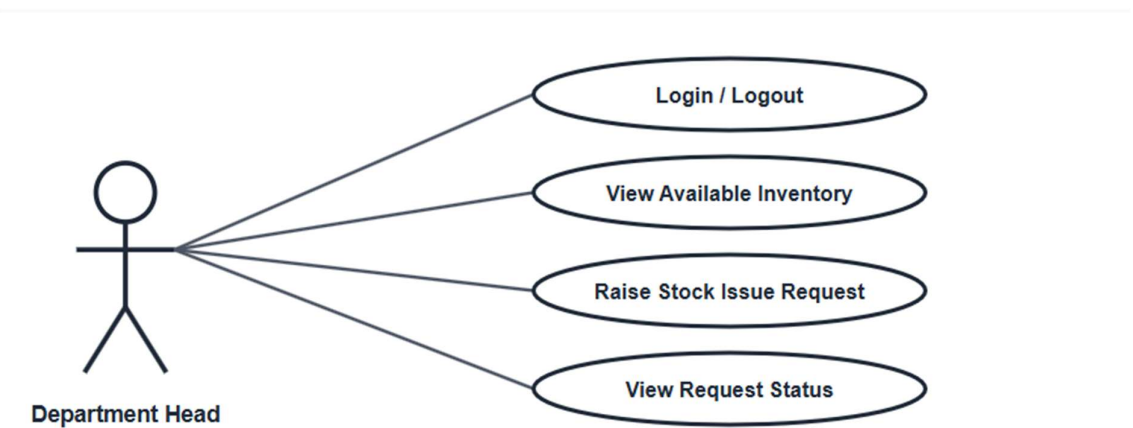


Fig 6.2.3: Department Head Module Use Case: Represents the functions available to the Department Head for requesting inventory items and tracking request status. It defines the interaction between the user and the system.

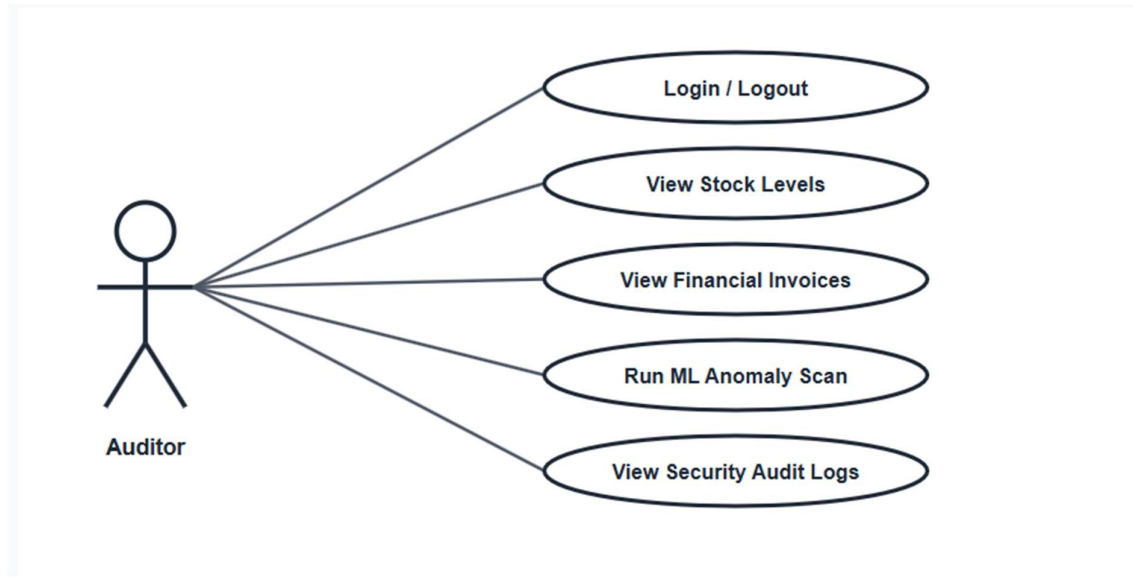


Fig 6.2.3: Auditor Module Use Case: Illustrates the Auditor's role in viewing audit logs, monitoring alerts, and analysing inventory activities. It highlights the read-only access provided for auditing purposes.

6.3 CLASS DIAGRAM

The Class Diagram displays the static structure of the StockWatch program. It shows the main code classes (User, InventoryItem, StockMovement, PurchaseOrder, Invoice, AuditLog), their internal fields, their methods, and how the classes relate to each other.

The StockWatch system architecture is structured around key classes: User and InventoryItem store profile and product details, while StockMovement, PurchaseOrder, Invoice, and AuditLog handle business transactions and security. Each class contains specific operations (methods)—such as user authentication (login()), stock management (addItem()), order procurement (receiveStock()), auto-billing (autoGenerateInvoice()), and database auditing (writeLog()). Structurally, a user manages multiple items that undergo movements, and purchase orders share a one-to-one relationship with generated invoices, with all operations generating audit log trails for secure traceability.

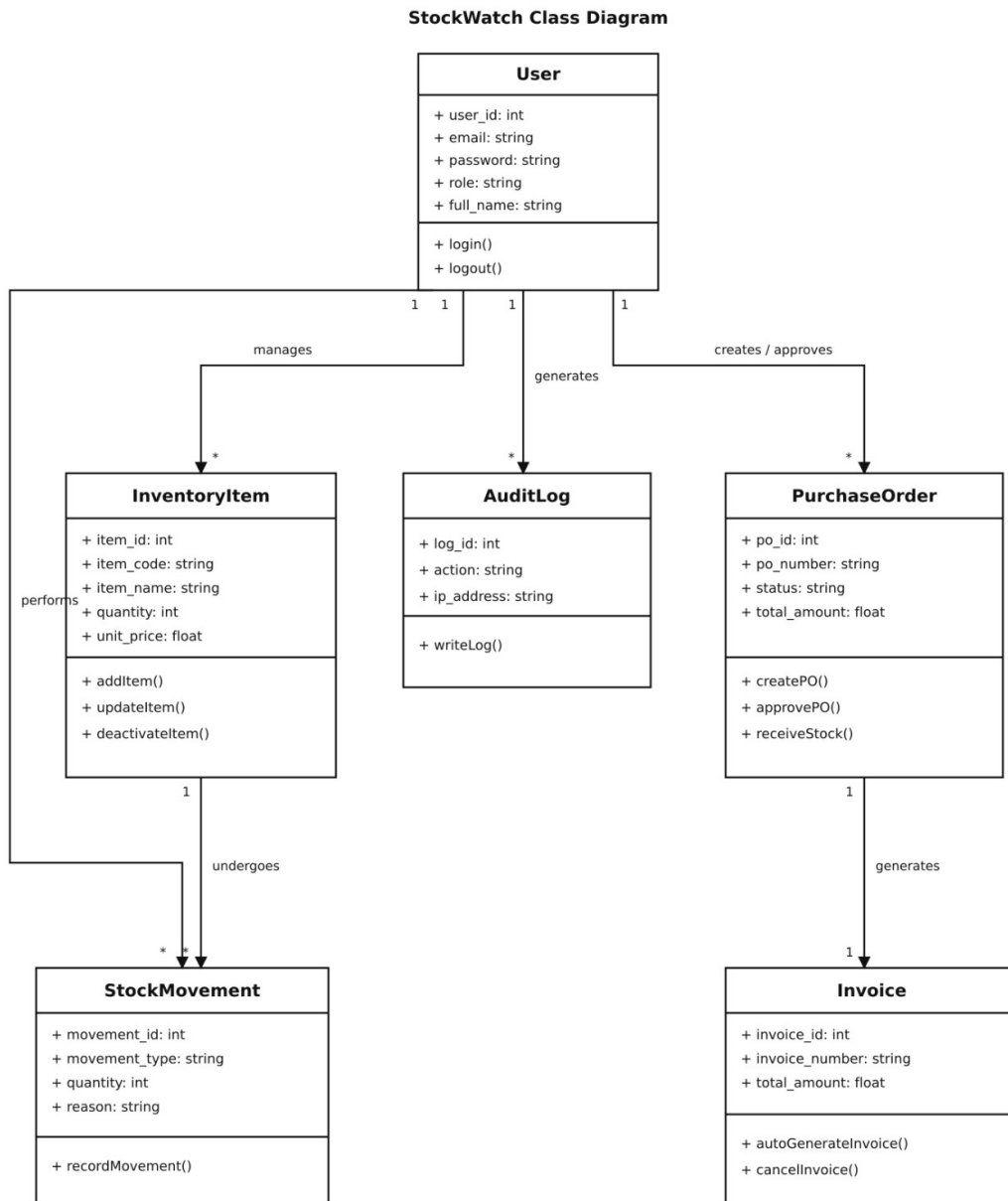


Fig 6.3: Class Diagram: Displays the classes, attributes, methods, and relationships used in the StockWatch system. It represents the object-oriented design of the application.

CHAPTER 7

IMPLEMENTATION

The implementation phase of the StockWatch system involves translating the system design models and architectural plans into functional code. The system is implemented as a lightweight, secure web application utilizing a modern two-layer split. The user interface runs in the client's web browser as a Single Page Application (SPA) built using HTML5, Vanilla CSS3, and JavaScript (ES6). The backend runs on a Python server using the Flask framework to handle the application logic, process security checks, and connect to the MySQL database. Security is enforced through JSON Web Tokens (JWT) that are signed on the server during login and saved in the client's local storage to identify the user's role and restrict access to page screens.

7.1 MODULES

7.2.1 Admin Module

This module is designed for system administrators to manage operator accounts, permissions, and security parameters. The Admin dashboard provides three core features:

- **User Directory Management:** Allows the Admin to create new user profiles (Full Name, Email, Password, Role, and Department). It validates that the email is unique, checks that the password meets complexity requirements, and saves the user account in the database.
- **Account Status Control:** Allows the Admin to block or unblock user accounts instantly. If an account is blocked, the user is immediately logged out and cannot log back in, which helps handle security incidents.
- **Security Logs Viewer:** Displays the history of all database edits, logins, and IP addresses, helping the admin trace all changes.

7.2.2 Inventory Manager Module

This module handles all core warehouse operations, catalogs, stock levels, purchasing, and approvals. It is used by the Inventory Manager and Admin:

- **Inventory Catalog Management:** Provides full CRUD (Create, Read, Update, Delete) features for items. The manager can input item names, codes, prices, min-stock levels, and categories. The system automatically creates a unique QR code string for each item.
- **Stock Movements Recorder:** Allows managers to record stock entries (IN), usages (OUT), transfers, and adjustments. It checks if the quantity is valid and updates the inventory table.
- **Procurement and Purchase Orders:** Allows managers to draft purchase orders, select suppliers, add items, and calculate totals. It requires another manager to approve the PO for security.
- **Receiving and Auto-Billing:** When order packages arrive, the manager inputs the received counts. The system updates inventory levels and automatically creates a completed invoice bill.

7.2.3 Department Head Module

This module is designed for staff members who need to request stock items for their specific labs or departments. The Department Head dashboard provides:

- **Item Directory Lookup:** Allows heads to browse the catalog, view current available stock, and check item details.
- **Stock Issue Request:** Allows heads to submit request forms specifying the item, quantity, and purpose. The request is logged in the approvals table as "Pending".
- **Request Status Tracker:** Shows the head if their request is approved, rejected, or pending.

7.2.4 Auditor Module

This module is designed for financial and safety reviews, offering full read-only access to logs and machine learning scans:

- **System Overview Dashboard:** Displays key inventory charts (like stock categories, values, and alert history).
- **Machine Learning Outlier Detector:** Allows auditors to trigger the Isolation Forest scan. It reads the stock movements history and flags suspicious edits (like unusually high stock drops) as outliers.
- **Audit Trails:** Displays security logs with timestamps, actions, and IP addresses for full compliance checking.

CHAPTER 8

SYSTEM TESTING

Software testing is a critical phase of the system development life cycle. The primary goal of testing is to verify that the StockWatch system is correct, reliable, and performs well under different conditions. Testing helps to identify errors, find hidden bugs, and ensure that the application behaves exactly as designed. The testing phase confirms that all security rules, role restrictions, inventory adjustments, and automated invoicing workflows perform correctly.

The core objectives of the system testing phase are:

- To verify that user login sessions are handled securely and credentials are encrypted.
- To validate that role-based access rules correctly restrict page screens.
- To ensure that stock calculations, thresholds, and invoices are generated without errors.
- To confirm that user entries are validated to block negative values and unauthorized actions.
- To verify that every user transaction writes a trace log with client IP addresses.

8.1 Testing Methods

The system testing phase involves checking the entire StockWatch application to ensure it operates correctly, satisfies all design requirements, and is free of programming bugs. We used four main testing methods:

- **Unit Testing:** This method tests individual pieces of code or database queries on their own. For example, we tested the Flask helper function that calculates the total cost of a purchase order to ensure it handles math correctly. We also tested user role validation checks to make sure they return true or false accurately depending on the user.
- **Integration Testing:** This method checks how different modules connect and exchange data. In an inventory app, this is very important. For example, when stock arrives and the manager records it, integration testing verifies that:

1. The item's stock level increases in the database.
 2. A stock transaction log is created in the history table.
 3. A billing invoice statement is automatically compiled in the database.
- **System Testing:** This method tests the complete application from the user's perspective. We opened the browser, logged in under different accounts, clicked buttons, filled out forms, and scanned test QR codes using the camera. This ensures that the frontend and backend servers communicate smoothly under realistic usage.
 - **User Acceptance Testing (UAT):** The system was tested by group members to ensure it meets the requirements of the project. We checked that the interface loads quickly, that role access is restricted properly, and that the alerts are triggered correctly when items fall below their minimum levels.

8.2 Validation and Verification

Verification checks that the software meets the technical specifications, while validation checks that the system produces the correct results when handling real user inputs.

- **Input Boundaries Validation:** The system blocks users from entering negative values in numeric fields. For example, if a user enters -10 in the "Add Item" price box, the system blocks the save operation.
- **Role Verification:** We verified that a user cannot bypass page security by typing URLs directly. If a Department Head attempts to open the Admin configuration page, the frontend router automatically redirects them to their own dashboard.
- **Workflow Security Verification:** We verified that a manager cannot approve a purchase order they created, preventing unauthorized expenditures.

8.3 TEST CASES

Test ID	Description	Expected Result	Final Result
TC_01	User Authentication	User should log in successfully and the dashboard should open.	Dashboard opened successfully.
TC_02	Invalid Login Attempt	System should display “Invalid email or password.”	Error message displayed successfully.
TC_03	Add Inventory Item Validation	System should reject negative price values.	Invalid input was rejected successfully.
TC_04	Stock Availability Check	System should prevent deduction when stock is insufficient.	Transaction was blocked successfully.
TC_05	Purchase Order Self-Approval Block	System should prevent users from approving their own purchase orders.	Self-approval was blocked successfully.
TC_06	Stock Receipt and Automatic Invoice	Stock should be updated and an invoice should be generated automatically.	Stock updated and invoice generated successfully.
TC_07	Departmental Issue Request	System should reject requests when the item is out of stock.	Request was blocked successfully.
TC_08	Isolation Forest Anomaly Detection	System should detect and display unusual stock transactions.	Anomalies were detected and displayed successfully.

CHAPTER 9

SNAPSHOTS

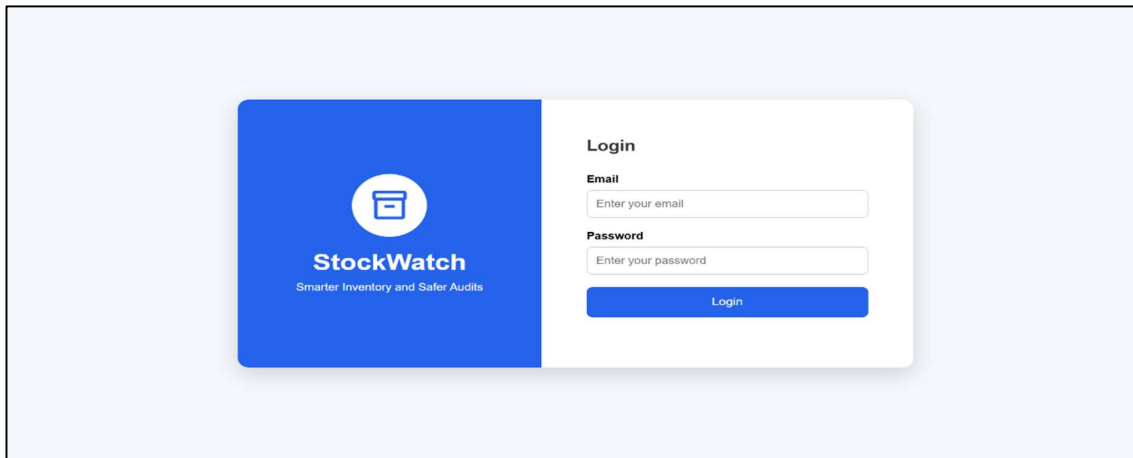


Fig 9.1: Login Page: Provides a secure login interface for users to access the system using their registered email and password. It verifies user credentials and grants role-based access.

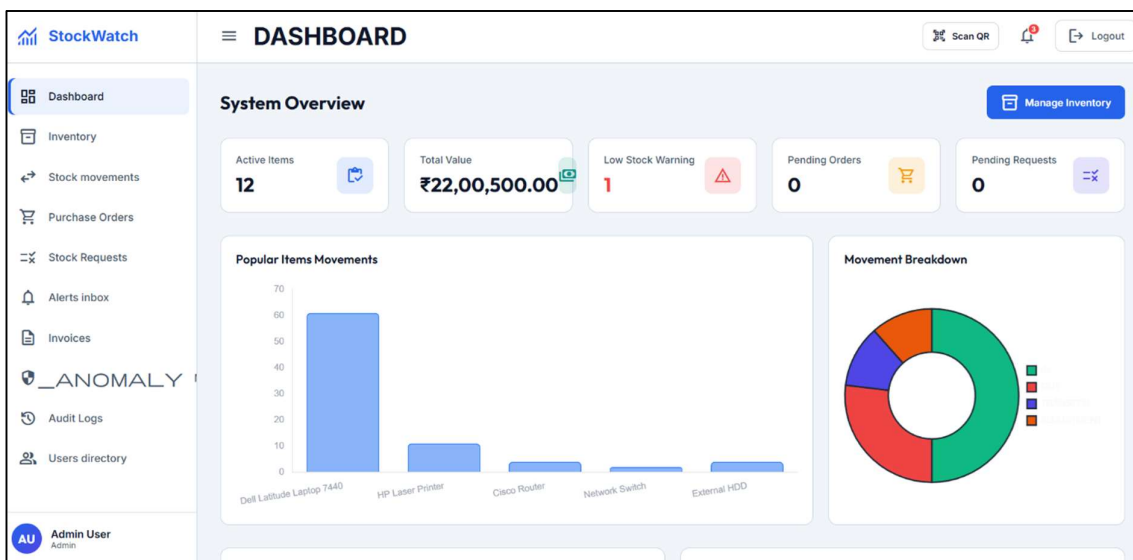


Fig 9.2: Dashboard: Displays an overview of inventory information, stock status, alerts, and system activities. It helps users quickly monitor important inventory details.

StockWatch

Dashboard

Inventory

Stock movements

Purchase Orders

Stock Requests

Alerts inbox

Invoices

ANOMALY

Audit Logs

Users directory

Admin User

INVENTORY

Scan QR

Logout

Inventory Directory

+ New Inventory Item

Inventory Items List

Search by name or code...

CODE	ITEM NAME	CATEGORY	STOCK	UNIT PRICE	TOTAL VALUE	DEPARTMENT	SUPPLIER	ACTIONS
17H004	24 Inch Monitor Monitor	Electronics	15 Nos	₹9,000.00	₹1,35,000.00	IT	TechNova Solutions	
17H003	Cisco Router Router	Networking	14 Nos	₹12,000.00	₹1,68,000.00	IT	Elite Electronics	
17H001	Dell Latitude Laptop 7440 Updated Laptop	Electronics	15 Nos	₹70,000.00	₹10,50,000.00	IT	TechNova Solutions	
17H010	External HDD Storage	Accessories	20 Nos	₹5,500.00	₹1,10,000.00	IT	TechNova Solutions	
17H002	HP Laser Printer Printer	Electronics	9 Nos	₹18,000.00	₹1,62,000.00	Warehouse	Global IT Supplies	
17H006	Keyboard Keyboard	Accessories	40 Nos	₹900.00	₹36,000.00	IT	Elite Electronics	
17H007	Network Switch	Networking	11 Nos	₹2,500.00	₹27,500.00	IT	Global IT	

Fig 9.3: Inventory Page: Allows users to add, update, search, and manage inventory items. It displays complete information about available stock in the system.

StockWatch

Dashboard

Inventory

Stock movements

Purchase Orders

Stock Requests

Alerts inbox

Invoices

_ANOMALY

Audit Logs

Users directory

AU

Admin User

Admin

STOCK

Scan QR

Logout

Stock Movements & Transactions

Record Stock Transaction

Transaction Type *

Stock IN (+)

Select Item *

-- Select Inventory Item --

Quantity *

1

Reason / Remark *

e.g. Inward batch / Issue to IT Dept

Execute Transaction

Movements Transaction Log

DATE/TIME	ITEM	TYPE	QTY	STOCK FLOW	CONTEXT	OPERATOR
22/7/2026, 6:23:45 pm	External HDD 2TH933	IN	2 Nos	18 → 20 (+2)	Received from PO PO1004 Purchase Ref #4	Admin User
22/7/2026, 6:23:45 pm	Office Chair 2TH909	IN	5 Nos	20 → 25 (+5)	Received from PO PO1004 Purchase Ref #4	Admin User
22/7/2026, 5:42:21 pm	Speaker 5K-438795	IN	8 Nos	0 → 8 (+8)	Initial stock entry Manual Ref #None	Admin User
14/7/2026, 3:29:45 pm	Cisco Router 2TH903	IN	1 Nos	13 → 14 (+1)	Received from PO PO1003 Purchase Ref #3	Asha Manager
14/7/2026, 3:29:07 pm	Network Switch 2TH907	IN	1 Nos	10 → 11 (+1)	Received from PO PO1003 Purchase Ref #3	Asha Manager
14/7/2026, 3:29:07 pm	Cisco Router	IN	1	12 → 13 (+1)	Received from PO PO1003	Asha

Fig 9.4: Stock Movements Page: Records all inventory transactions such as stock addition, removal, transfer, and adjustment. It helps maintain accurate stock movement history.

StockWatch

Dashboard

Inventory

Stock movements

Purchase Orders

Stock Requests

Alerts inbox

Invoices

ANOMALY

Audit Logs

Users directory

Admin User

PURCHASE

Scan QR

9

Logout

Procurement & Purchase Orders

New Purchase Order

Purchase Orders Registry

Search by PO number or supply

PO NUMBER	SUPPLIER	ORDER DATE	EXPECTED DELIVERY	TOTAL AMOUNT	STATUS	CREATED BY	APPROVED BY	ACTIONS
PO-2026-0006	Global IT Supplies	Tue, 14 Jul 2026 00:00:00 GMT	Sat, 15 Aug 2026 00:00:00 GMT	₹45,000.00	Approved	Admin User	Asha Manager	
PO1005	Smart Trade Distributors	Thu, 18 Jun 2026 00:00:00 GMT	Thu, 25 Jun 2026 00:00:00 GMT	₹13,000.00	Cancelled	Asha Manager	Admin User	
PO1004	Prime Office Systems	Mon, 15 Jun 2026 00:00:00 GMT	Sat, 20 Jun 2026 00:00:00 GMT	₹17,500.00	Received	Asha Manager	Admin User	
PO1003	Elite Electronics	Wed, 10 Jun 2026 00:00:00 GMT	Mon, 15 Jun 2026 00:00:00 GMT	₹24,000.00	Received	Kiran IT	Admin User	
PO1002	Global IT Supplies	Mon, 08 Jun 2026 00:00:00 GMT	Fri, 12 Jun 2026 00:00:00 GMT	₹36,000.00	Ordered	Asha Manager	Admin User	
PO1001	TechNova Solutions	Mon, 01 Jun 2026 00:00:00 GMT	Fri, 05 Jun 2026 00:00:00 GMT	₹1,30,000.00	Received	Asha Manager	Admin User	

Fig 9.6: Purchase Order Page: Enables users to create, manage, and monitor purchase orders for inventory items. It supports the procurement process from order creation to approval.

StockWatch

Dashboard

Inventory

Stock movements

Purchase Orders

Stock Requests

Alerts inbox

Invoices

ANOMALY

Audit Logs

Users directory

AU

Admin User

Admin

APPROVALS

Scan QR

1

Logout

Stock Requests & Approvals

Raise Stock Request

Stock Issues & Requests Queue

REQ ID	ITEM REQUESTED	AVAILABLE STOCK	QTY REQUESTED	PURPOSE / DEPARTMENT	STATUS	REQUESTED BY	APPROVED BY	ACTIONS
REQ-1	Dell Latitude Laptop 7440 170001	15	2	Project Date: 20/6/2026	Approved	Ravi Head	Admin User "OK"	Closed
REQ-2	HP Laser Printer 170002	9	1	Office Date: 20/6/2026	Approved	Ravi Head	Asha Manager "Approved"	Closed
REQ-3	Wireless Mouse 170003	50	5	Replacement Date: 20/6/2026	Approved	Asha Manager	Admin User "Approved"	Closed
REQ-4	UPS 170004	9	1	Maintenance Date: 20/6/2026	Rejected	Kiran IT	Admin User "Budget"	Closed
REQ-5	Office Chair 170005	25	2	Expansion Date: 20/6/2026	Rejected	Ravi Head	Asha Manager "Rejected due to insufficient details"	Closed

Fig 9.7: Stock Requests page: Allows department heads to submit requests for required inventory items. It also displays the current status of each request.

ALERTS

Notifications & System Alerts

System Alerts Inbox

RECEIVED TIME	CATEGORY	SEVERITY	ALERT DETAILS	STATUS	ACTION
22/7/2026, 6:28:58 pm	Anomaly	High	Anomaly detected — Movement ID 15 for 'Dell Latitude Laptop 7440': IN of 20 units (Prev: 30, New: 50) Item: ITH001 Warehouse Stock: 15	Unread	
22/7/2026, 6:28:58 pm	Anomaly	High	Anomaly detected — Movement ID 18 for 'Dell Latitude Laptop 7440': ADJUSTMENT of 30 units (Prev: 45, New: 15) Item: ITH001 Warehouse Stock: 15	Unread	
22/7/2026, 6:28:58 pm	Anomaly	Medium	Anomaly detected — Movement ID 24 for 'Speaker': IN of 8 units (Prev: 0, New: 8) Item: SH-428795 Warehouse Stock: 8	Unread	

Fig 9.8: Alerts Inbox Page: Displays important notifications such as low-stock warnings and anomaly alerts. It helps users take timely action to maintain inventory levels.

INVOICES

Financial Invoices

Invoices Records Ledger

INVOICE NUMBER	SUPPLIER	ORDER PO REF	TOTAL BILL AMOUNT	BILLING STATUS	DATE GENERATED	BILLED BY	ACTIONS
INW-2026-0002	Prime Office Systems	PO1004	₹17,500.00	Finalized	22/7/2026, 6:23:45 pm	Admin User	
INW-2026-0001	Elite Electronics	PO1003	₹24,000.00	Finalized	14/7/2026, 3:29:45 pm	Admin User	

Fig 9.9: Invoice Page: Shows invoices generated after successful stock receipt and purchase completion. It maintains financial records related to inventory purchases.

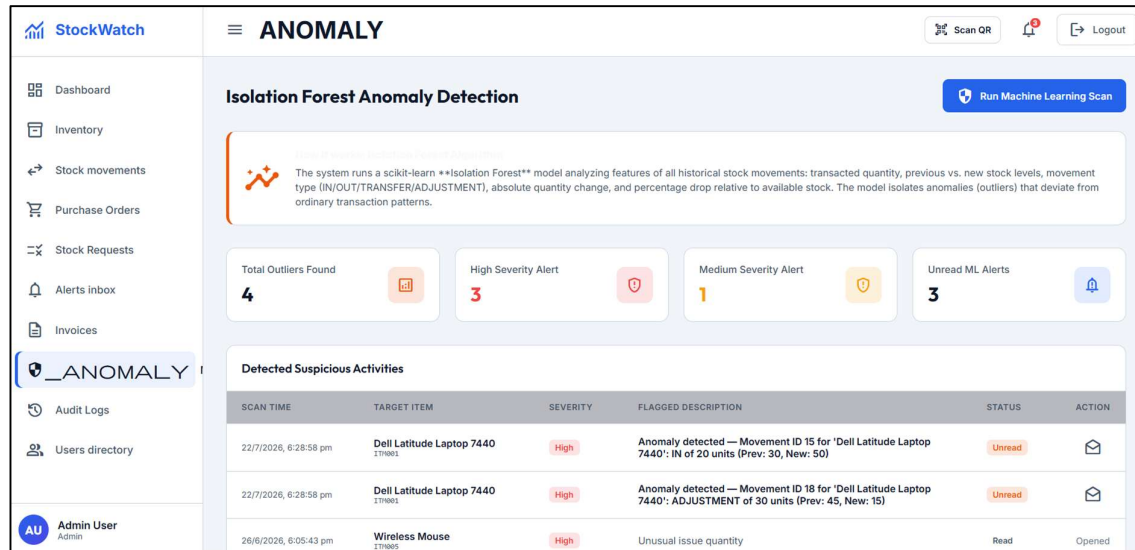


Fig 9.10: Anomaly Detection Page: Displays unusual inventory transactions identified using the Isolation Forest algorithm. It helps detect suspicious stock activities for further investigation.

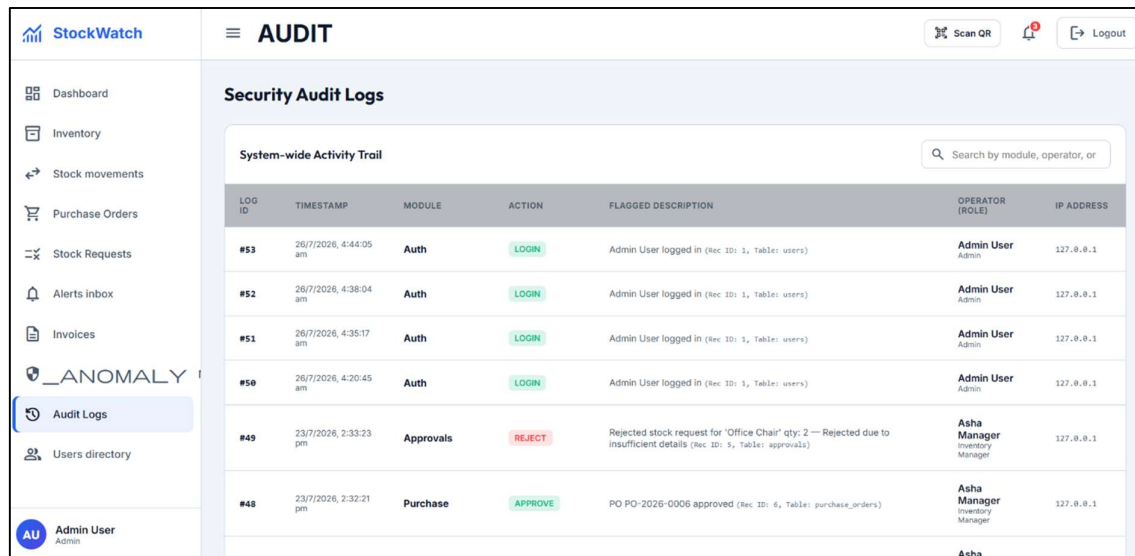


Fig 9.11: Audit Logs Page: Records all user activities performed within the system along with timestamps and other details. It improves accountability and supports auditing of inventory operations.

CONCLUSION

In conclusion, the design, implementation, and testing of the StockWatch system successfully address the major issues associated with manual inventory registers and paper-based tracking methods. By providing a secure, web-based platform, the project reduces human errors in data recording, speeds up stock counting through QR codes, and introduces automation into procurement. The objective of delivering "Smarter Inventory and Safer Audits" is achieved by integrating user dashboards with a relational database, creating a reliable digital inventory management system.

From a technical perspective, the project demonstrates the effectiveness of a three-tier architecture consisting of the presentation, application, and database layers. The frontend, developed using HTML, CSS, and JavaScript, provides a responsive user interface, while the Python Flask backend manages business logic, JWT-based session security, and form validation before storing data in the MySQL database. The database ensures data integrity and maintains consistent stock records across all departments and users.

A key achievement of StockWatch is its focus on security and auditability. All system activities are recorded in audit logs with the user's name, timestamp, action performed, and client IP address, creating a secure and traceable audit trail. The integration of the Isolation Forest machine learning model further improves security by analysing stock movements and identifying unusual activities such as abnormal stock reductions or suspicious transfers.

This project has provided valuable practical experience in full-stack web development, database design, role-based access control, and machine learning integration. The completed system can be effectively used in educational institutions, offices, and small warehouses to replace manual inventory tracking.

The modular architecture also supports future enhancements such as barcode scanner integration, automated supplier notifications, and biometric authentication. Overall, StockWatch demonstrates that a modern, secure, and cost-effective software solution can significantly improve inventory management, auditing, and operational efficiency.

FUTURE ENHANCEMENT

The modular design of the StockWatch application ensures that it can be easily expanded to support new technologies. Four major future upgrades are planned for the next versions:

- **Automated Supplier Notification (Email/SMS Alerts):** Currently, when items fall below the minimum stock limit, the manager receives a dashboard alert, but must still manually place the reorder. In the future, the application layer will connect to an email gateway (like SendGrid) or SMS gateway (like Twilio). As soon as stock drops below the threshold, the system will automatically draft and email a pre-formatted purchase order directly to the assigned supplier, minimizing procurement delays.
- **Hardware-Based Biometric Verification:** To further secure high-value stock transfers and hardware issues, we plan to integrate biometric verification mechanisms. Currently, users sign off on transactions using passwords and JWT session authentication. By introducing USB fingerprint scanners or camera-based facial recognition logins, the system can guarantee the physical presence of the operator during key movements. This will eliminate password-sharing and make the security audit logs even more reliable.
- **Multi-Branch Warehouse Synchronization:** Another important upgrade is adding support for multi-branch warehouse systems. StockWatch is currently designed to manage inventory for a single local warehouse or university campus. In the future, the database schema will be updated to include branch codes. This will allow a central administration panel to monitor stock levels, view purchase order trends, and coordinate item transfers between multiple warehouse branches located in different cities, all in real-time.
- **Predictive Analytics and Machine Learning Forecasting:** Lastly, we plan to expand the machine learning module to include predictive stock forecasting. While the current system uses the Isolation Forest model to detect suspicious log entries, future versions will use regression and time-series forecasting models (such as ARIMA or LSTM networks). By analyzing historical transaction patterns, the system will be able to predict exactly when a specific item is likely to run out based on seasonal demand or lab calendars, helping managers order items proactively.

REFERENCES

- [1] **Merle Hewing, Yuanchen Zhao, Manuel Vossel, Paul Nieschler, Tobias Kleinert,** (2025), **"A Data-Driven Framework for Anomaly Detection in Industrial Systems Using Log Data"**, 51st Annual Conference of the IEEE Industrial Electronics Society (IECON 2025), pp. 1-6.
- [2] **R. Sekar, Hanke Kimm, Rohit Aich,** (2024), **"eAudit: A Fast, Scalable and Deployable Audit Data Collection System"**, 2024 IEEE Symposium on Security and Privacy (SP), pp. 3571-3589.
- [3] **Kulasekhara Reddy Kotte,** (2025), **"AI-Driven Framework for Real-Time Financial Anomaly Detection and Transparent Audit Trail Generation in ERP Systems"**, 3rd International Conference on Intelligent Cyber Physical Systems and Internet of Things (ICoICI 2025), pp. 587-594.
- [4] **Michael Siek, Jefferson Richard Djaya,** (2025), **"QR Based Web Application For Warehouse Management System"**, 15th International Conference on Information & Communication Technology and System (ICTS 2025), pp. 1-6.
- [5] **Sunaryo Winardi, Arifin, Ng Poi Wong, Apriyanto Halim, Sunario Megawan,** (2024), **"Enhancing Warehouse Inventory Management through IoT Tools for Monitoring Stock Items"**, 2nd International Conference on Technology Innovation and Its Applications (ICTIIA 2024), pp. 1-6.
- [6] **Nadun Wijesinghe, Hadi Hemmati,** (2023), **"Log-based Anomaly Detection of Enterprise Software: An Empirical Study"**, 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS), pp. 12–23, DOI: 10.1109/QRS60937.2023.00012.
- [7] **Gunther Misahuaman, Alfredo Daza, Emily Zavaleta,** (2021), **"Web-based Systems for Inventory Control in Organizations: A Systematic Review"**, 2021 IEEE/ACIS 22nd International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD), pp. 15–20. DOI: 10.1109/SNPD51163.2021.9704993.

